

1. Veiledet læring

1.1 Data

Kvalitative eller kvantitative variabler:

- Kvalitative representerer kategorier.
 - Sjanger, aldersgruppe, type frukt ...
- Kvantitative representerer numeriske verdier:
 - inntekt, alder, antall lyttere, areal, nedbørsmengde, temperatur...

Eksempel Titanic datasett:

Table 3: Titanic-datasettet etter at vi har valgt ut hvilke variable vi vil beholde, og innført notasjon.

	x_1	x_2	x_3	y
$\mathbf{x}^{(1)}$	3	male	22	0
$\mathbf{x}^{(2)}$	1	female	38	1
$\mathbf{x}^{(3)}$	3	female	26	1
$\mathbf{x}^{(4)}$	1	female	35	1
$\mathbf{x}^{(5)}$	3	male	35	0
$\mathbf{x}^{(6)}$	3	male		0
$\mathbf{x}^{(7)}$	1	male	54	0
$\mathbf{x}^{(8)}$	3	male	2	0
$\mathbf{x}^{(9)}$	2	male	27	1

1.2 Logistisk regresjon

Enkleste modellen er lineær regresjon:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}\mathbf{x} + b$$

Kan gjøres om til sansynlighetsfordeling (mellom 0 og 1) ved å sette inn for `#Sigmoid-funksjonen` (også kjent som `#aktiveringsfunksjon`)

$$\sigma(z) = \frac{1}{1 + e^{-z}} \implies y_{pred} = \frac{1}{1 + e^{-(\sum_{i=1}^n w_i x_i + b)}}$$

1.2.1 Trening og tapsfunksjon

Funksjonen som brukes til å beregne målopnnåelse kalles for `#tapsfunksjon`. Forteller hvor nærme modellens prediksjon er den riktige verdien, eller target:

$$\mathcal{L}(y_{pred}, y)$$

Den tar inn modellprediksjonene y_{pred} og `#targets` y . **NB!** må være deriverbar (se 1.2.2 Gradient descent)

Vi vil finne en `#likelihood` og ikke en sannsynlighet fordi vi **ikke** har lyst til å endre dataene slik at sannsynligheten for target endrer seg. men heller endre modellen slik at den estimerte sannsynligheten passer med den virkelige (target).

Vi ønsker å ha likelihood for å estimere riktig være maksimal, så vi kan legge til et minustegn på tapsfunksjonen for å minimere denne. Vi bruker gjerne å ta logaritmen av av likelihood for å f" **log-likelihood** (Minimere tap == maksimere Likelihood). Dette kalles for `#cross-entropy-loss`.

Eksempel på tapsfunksjon:

Vektet tapsfunksjon: Maksimere likelihood for riktig kategorisering av dataene

$$\mathcal{L}(y_{pred}, y) = -\omega_1 y \ln y_{pred} - \omega_0 (1 - y) \ln(1 - y_{pred})$$

Der ω_0 og ω_1 er vekter for klassene, se [Vektet tapsfunksjon](#).

y er target og vi kan bruke y_{pred} fra [formelen her](#).

Vi kan nå skrive om modellen definert av parametrene $\mathbf{w}$ og b for å finne parametrene som minimerer tapet i gjennomsnitt over alle datapunktene (x). Dette kaller vi for å **trene modellen**. Vi får da at vi skal maksimere log-likelihooden for parametrene som vi definerer:

$$\theta = (\mathbf{w}, b) \implies \hat{\theta} = \arg_{\theta} \min \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$$

Dette kan løses med bruk av [gradient descent](#).

1.2.2 Gradient descent

Når vi ser på funksjonen vår beveger vi oss i rommet spent ut av verdiene θ , fire verdier gir et fire dimensjonalt rom. Vi vil finne minimumspunktet til funksjonen uten at vi kjenner formen til den. **NB!** så lenge tapsfunksjonen er konveks, så vil vi alltid kunne finne lokalt minium med gradienten.

I starten av treningen har alle parametrene en tilfeldig verdi, og vi oppdaterer verdiene iterativt, gjennom flere steg. Gradienten til tapsfunksjonen peker i retning av økende tap, og siden vi ønsker å minke tapet må vi bevege oss i motsatt retning. Dette representerer vi gjennom et minustegn, og regelen for parameteroppdateringen kan skrives som følger:

$$\theta^{t+1} = \theta^t - \eta \frac{\partial}{\partial \theta} \mathcal{L}(f(\mathbf{x}; \theta), y)$$

Der η er `#læringsraten`, som bestemmer hvor mye hvert steg t skal korrigere til parameterverdien. **NB!** η eller læringsraten er en parameter for å justere treningen til modellen, men er **ikke** en del av modellen! Dette kaller vi for en `#hyperparameter`, som definerer læringsprosessen, men ikke modellen.

Framgangsmåten vil da bli å finne gradienten av tapsfunksjonen, med hensyn til vær av parametrene. Eksempel med titanic modellen:

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} [-y \ln \sigma(\mathbf{w}\mathbf{x} + b) - (1 - y) \ln (1 - \sigma(\mathbf{w}\mathbf{x} + b))] \\
 &= -y \frac{1}{\sigma(\cdot)} \frac{\partial}{\partial w_j} \sigma(\mathbf{w}\mathbf{x} + b) + (1 - y) \frac{1}{1 - \sigma(\cdot)} \frac{\partial}{\partial w_j} (1 - \sigma(\mathbf{w}\mathbf{x} + b)) \\
 &= \frac{-y(1 - \sigma(\cdot)) + (1 - y)\sigma(\cdot)}{\sigma(\cdot)(1 - \sigma(\cdot))} \frac{\partial}{\partial w_j} \sigma(\mathbf{w}\mathbf{x} + b) \\
 &= \frac{\sigma(\cdot) - y}{\sigma(\cdot)(1 - \sigma(\cdot))} \sigma(\cdot)(1 - \sigma(\cdot)) \frac{\partial}{\partial w_j} (\mathbf{w}\mathbf{x} + b) \\
 &= (\sigma(\mathbf{w}\mathbf{x} + b) - y) x_j \\
 &= (y_{\text{pred}} - y) x_j \\
 \frac{\partial \mathcal{L}}{\partial b} &= \dots \\
 &= (\sigma(\mathbf{w}\mathbf{x} + b) - y) \frac{\partial}{\partial b} (\mathbf{w}\mathbf{x} + b) \\
 &= (y_{\text{pred}} - y) .
 \end{aligned}$$

1.2.3 Trening

En måte å strukturere `#læringsalgoritme` på:

```

class LogisticRegression:
    def __init__(self, learning_rate=0.1, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights,
        self.bias = None, None
        self.losses, self.train_accuracies = [], []

    def sigmoid_function(self, x):

    def _compute_loss(self, y, y_pred):

    def compute_gradients(self, x, y, y_pred):

    def update_parameters(self, grad_w, grad_b):

    def accuracy(true_values, predictions):
        return np.mean(true_values == predictions)

    def fit(self, x, y):
        self.weights = np.zeros(x.shape[1]) #x.shape = datapunkter, features
        self.bias = 0

    # Gradient Descent
    for _ in range(self.epochs):
        lin_model = np.matmul(self.weights, x.transpose()) + self.bias
        y_pred = self._sigmoid(lin_model)
        grad_w, grad_b = self.compute_gradients(x, y, y_pred)
        self.update_parameters(grad_w, grad_b)

        loss = self._compute_loss(y, y_pred)
        pred_to_class = [1 if _y > 0.5 else 0 for _y in y_pred]
        self.train_accuracies.append(accuracy(y, pred_to_class))
        self.losses.append(loss)

    def predict(self, x):
        lin_model = np.matmul(x, self.weights) + self.bias
        y_pred = self._sigmoid(lin_model)
        return [1 if _y > 0.5 else 0 for _y in y_pred]

```



Når skal bruke koden, bør en **ikke** bruke hele datasettet til å trene modellen. Del dataen opp i `#testdata` og `#treningsdata`. Treningsdataene brukes til parametertilpasning, mens testdataene ikke røres før modellen er ferdig tilpasset, og brukes for å evaluere modellen før denne tas i bruk. Bruk gjerne `train_test_split` fra biblioteket `scikit-learn`.

Eksempel på kjøring av kode:

PYTHON

```
# Training
train_epochs = 30

# Initialize and train the model
log_reg = LogisticRegression(learning_rate=0.01, epochs=train_epochs)
log_reg.fit(X_train, y_train)

# Make predictions
predictions = log_reg.predict(X_test)
```



1.2.4 Evaluering

Lurt å plote tapet (loss) mot treffsikkerhet (accuracy) for å se treningsprosedyren er god.

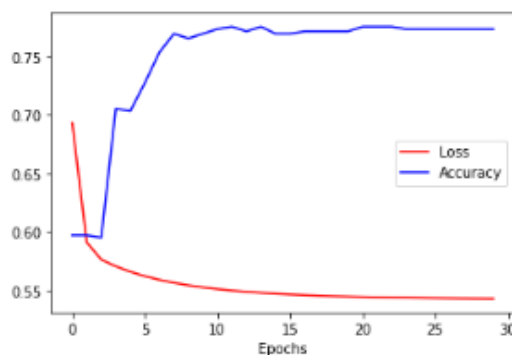


Figure 4: Eksempel på plott som viser tap (rød) og treffsikkerhet (blå) over epokene i en treningsprosedyre.

Mange metrikker baseres på `#confusion_matrix` som brukes ofte for å evaluere klassifiseringsmodeller.

		Predicted	
		Negative (PN)	Positive (PP)
True	Negative (N)	True negative	False positive
	Positive (P)	False negative	True positive

Figure 5: Confusion matrix for binær klassifisering.

Dette kan vi bruke til å regne ut flere matriser som i tabellen under:

Table 4: Et utvalg metrikker vi kan beregne basert på verdiene i confusion matrix.

Betegnelse	Uttrykk	Alternative navn
True Positive Rate (TPR)	$\frac{TP}{P} = \frac{TP}{TP+FN}$	Sensitivitet, recall, sannsynlighet for deteksjon
True Negative Rate (TNR)	$\frac{TN}{N} = \frac{TN}{TN+FP}$	Spesifisitet
False Positive Rate (FPR)	$\frac{FP}{N} = \frac{FP}{FP+TN}$	Sannsynlighet for falsk alarm, (1 - spesifisitet)
Positive Predictive Value (PPV)	$\frac{TP}{TP+FP}$	Precision

1.2.5 Klassifiseringsterskel

Enkel måte å justere på klassifiseringsterskelen er å endre linjen i `fit` og `predict` funksjonene:

```
[1 if _y > K else 0 for _y in y_pred]
```

Der K er klassifiseringsterskelen. Hvis en vil være sikker på at modellens predikasjon er riktig, vil en velge en høyere verdi for klassifiseringsterskelen som 0.7.

Det er vanlig å plote **FPR** (False Positive Rate) og **TPR** (True Positive Rate) som funksjon av klassifiseringsterskelen. Dette gir oss **Receiver Operating Characteristic** (`#ROC`)-kurven. Området under kurven til ROC kaller vi for `#AUC` (area under curve) som vi ønsker skal være større enn 0.5 (tilsvarer tilfeldig gjetting). ROC AUC representerer sannsynligheten for at modellen vil predikere en høyere verdi for et tilfeldig valgt positivt datapunkt enn for et tilfeldig valgt negativt datapunkt.

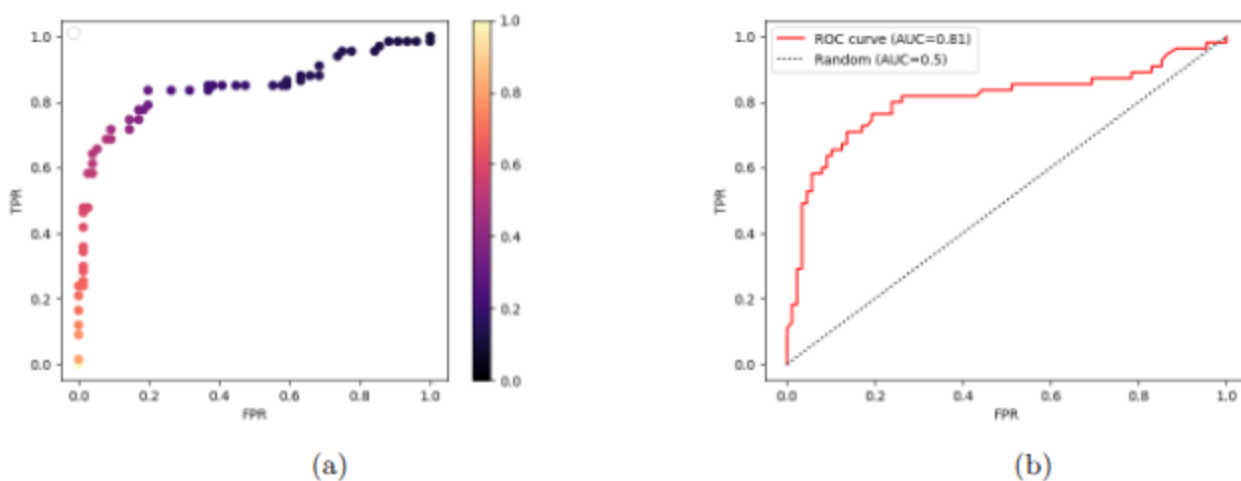


Figure 6: ROC-kurven, som viser FPR og TPR for ulike klassifiseringsterskler.

En annen vanlig kurve å studere er den som viser precision (TPR) og recall (PPV).

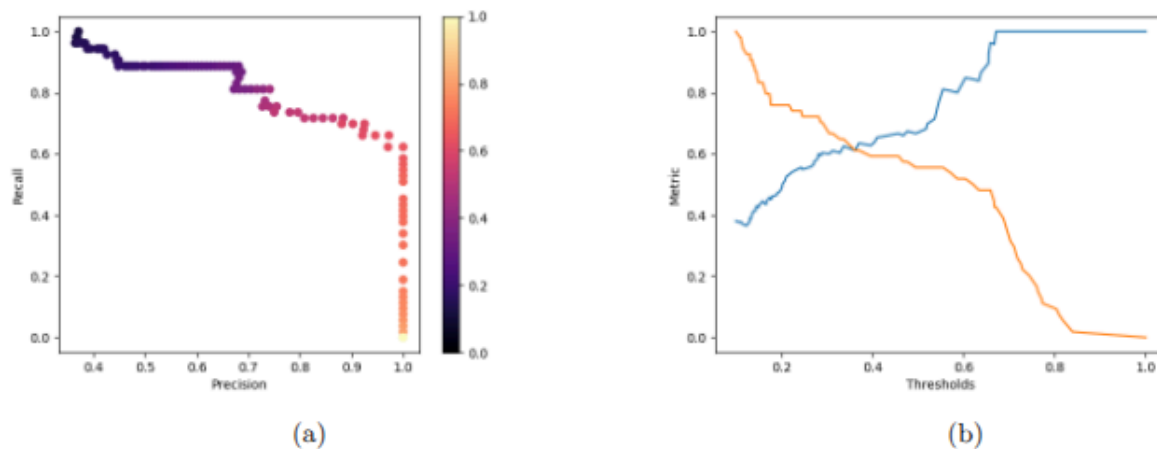


Figure 7: (a) Precision-recall-kurven , og (b) henholdsvis precision- og recall-kurven, for ulike klassifiseringsterskler.

Presisjon: Angir andel korrekt predikert positive per predikert positiv.

- Høy presisjon svarer til lavt relativt antall falske alarmer.

Recall: Angir andell korrekt predikert positive per totalt psotive.

- Høy recall svarer til lavt antall missed cases.

1.2.6 Kort om entropi og cross entropy loss

I informasjonsteori er entropi (også kalt Shannon entropy) en måte å kvantisere usikkerheten eller mengden informasjon tilgjengelig i en variabels mulige tilstander:

$$H(p) = - \sum_i p(x_i) \log p(x_i). \quad (28)$$

I tilfellet med Titanic-dataene representerer $p(x_i)$ her sannsynligheten for **Survived**. Kryss-entropien (cross entropy) representerer da forskjellen mellom to ulike sannsynlighetsfordelinger p og q :

$$H(p, q) = - \sum_i p(x_i) \log q(x_i). \quad (29)$$

Hvis vi sammenligner dette med `#tapsfunksjon` og `#cross-entropy-loss` , ser vi at den ene sannsynlighetsfordelingen representerer labels i datasettet, mens den andre representerer modellens prediksjoner. Tapsfunksjonen forteller oss altså om forskjellen mellom de to fordelingene, og siden målet vårt er å lage en klassifiseringsmodell, ønsker vi at forskjellen mellom de to fordelingene er minst mulig. Vi kan generalisere uttrykket over til:

$$\mathcal{L}(\hat{y}, y) = - \sum_i y_i \log(\hat{y}_i),$$

hvor summen går over alle klassene i .

1.2.7 Bayes' teorem

Vi kan bruke Bayes' teorem til å regne:

$$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{p(x)}$$

Vi får da at:

- $p(C_k)$ er *a priori* sannsynligheten for klasse, altså fordelingen av klassene i treningsdataene. Denne omtales som prior, fordi det er sannsynligheten for **klassetilhørighet** vi kan estimere uten å vite noe om det aktuelle datapunktet (legg merke til at x ikke forekommer i dette leddet).
- $p(x)$ er fordelingen av selve dataene, uavhengig av klasse (legg merke til at C_k ikke forekommer i dette leddet). Dette er den faktiske sannsynlighetsfordelingen av alle feature-verdiene for alle dataene, og den har like mange dimensjoner som datasettet har features. Dette leddet omtales som evidens.
- $p(x|C_k)$ er den betingede sannsynlighetsfordelingen av dataene gitt klassen, og det er en likelihood. Det er altså en egen sannsynlighetsfordeling – med like mange dimensjoner som datasettet har features – for hver klasse.

Vi kan regne $p(C_k)$, og for titanic casen kan vi se at $p(x)$ vil være vanskelig å bergne, men at den bidrar like mye til begge klassene så vi kan se vekk i fra den og få at:

$$p(C_k) \propto p(x|C_k)p(C_k)$$

Hvis vi antar at verdiene til featuresene er uavhengige av hverandre for alle features, så kan vi skrive dette om til Naiv Bayes:

$$p(C_k|x) \propto p(C_k) \prod_{i=1}^n p(x_i|C_k)$$

Som gir at den tilsvarende sannsynligheten for klassetilhørighet:

$$\hat{y} = \arg_{k \in [1, \dots, n]} \max p(C_k) \prod_{i=1}^n p(x_i|C_k)$$

1.2.8 Dimensjonsforbannelsen

FINN NOTATER HER

1.2.9 Trening på ubalansert data

Vektet tapsfunksjon